ASSAR Insurance Pricing & Information Engine

Technical Report: Architecture, Data Design, Retrieval, and Audit

This report documents the ASSAR pricing-and-information system end to end: what it does, how it is built, the design choices behind the data and retrieval layers, and a full audit table mapping every SQL table back to its source table and page in the manual. The source throughout is the Association of Insurers of Rwanda (ASSAR) Approved General Business Pricing Manual for the Rwandan Insurance Industry, Version 3, effective 25 January 2021 (81 pages).

---

1. Introduction

Insurance pricing manuals are written for people to read, not for machines to query. The ASSAR manual interleaves dense numeric rate tables, a 104-row fire grid, transit and marine commodity grids, liability and engineering rates, bonds, political-violence rates, and registers of large risks, with paragraphs of underwriting prose: definitions, conditions, warranties, exclusions, and guidance. A reader who wants a single number has to find the right page; a reader who wants to understand a clause has to read several.

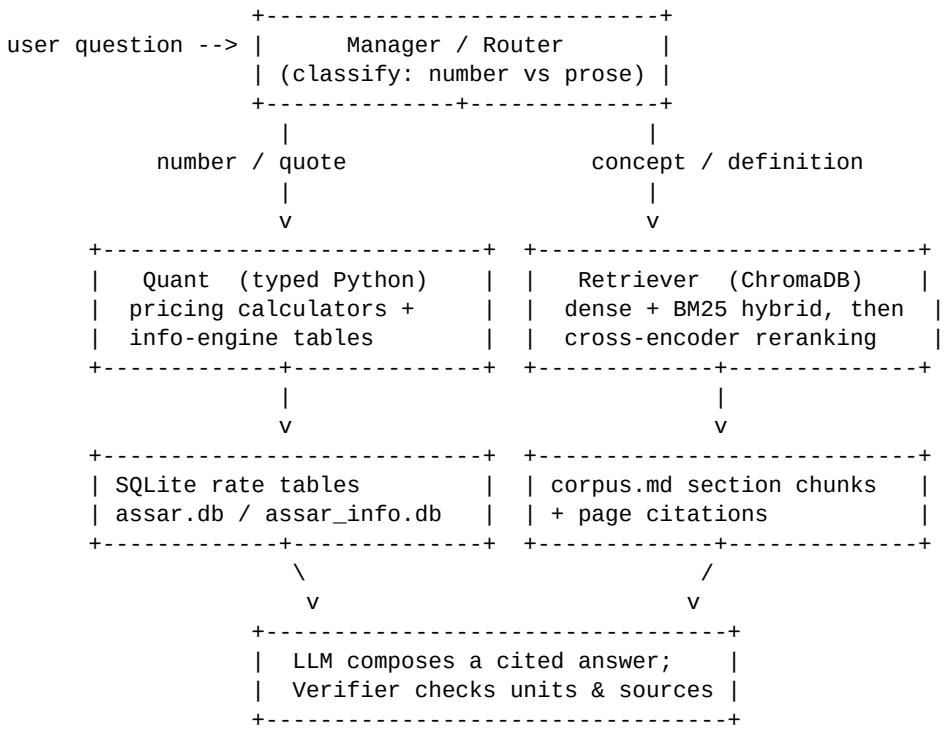
This system turns that manual into something a client or underwriter can query in plain language. It answers two distinct kinds of question. The first is quantitative and exact: what minimum rate applies to a risk, what multiplier applies for an indemnity period, what the mandatory excess is, what premium a given cover costs. The second is qualitative: what a term means, what is excluded, which warranty must be incorporated. The system is deliberately built so that the exact numbers are served exactly and deterministically, while the open-ended questions are answered fluently and with citations.

The result is a conversational assistant with three faces: a chat that holds a multi-turn conversation, a structured quote calculator, and a browsable database. All of it is grounded in the manual, and every premium is computed by tested code rather than guessed by a language model.

2. System Architecture

The architecture follows one principle: numbers and prose are stored and retrieved differently, and a language model orchestrates but never computes.

A free-text question first reaches a Manager/router that classifies it. If it is a pricing or number request, it is sent to the Quant layer: typed Python calculators that read exact rates from SQLite and compose the premium. If it is a table or comparison request, the relevant table in the per-table information database is matched and rendered directly from SQL. If it is a concept question, it is sent to the Retriever, which performs hybrid search (dense vectors plus BM25 keywords) with cross-encoder reranking over the manual's prose. The language model then composes a final answer with page citations; a grounding check enforced in code ensures no rate or amount reaches the user unless it appears in a calculator result or a retrieved passage, so the unit of each figure (percent vs per-mille vs franc amount) cannot be silently invented.



The deliberate trade-off is determinism and auditability over end-to-end neural generation. A purely generative system would be simpler but impossible to trust for binding figures, because embeddings blur exact values and a model reading a

rate out of retrieved text can round or transcribe it wrong. By confining the model to understanding the request, selecting a tool, and phrasing the result, the financially sensitive arithmetic stays under tested, reproducible control. A separate quote path bypasses the model entirely and calls the calculators directly, so pricing works even with no language-model key configured.

The language model is pluggable (Groq, Ollama, or Hugging Face via one OpenAI-compatible client). Embeddings always run locally. Premiums are produced by typed functions covered by 39 deterministic tests.

---

3. Data Design Choices

The system keeps two SQLite databases, both built from the same manual but shaped for different consumers. This is the central data decision.

The first, assar.db, uses four generic tables. A single rate table holds most per-category rates, namespaced by a scheme column (fire, public_liability, aviation, bond, pvt, and so on). Separate tables hold the two-dimensional transit commodity grids, the ordered discount and multiplier schedules, and per-product constants such as minimum premiums and mandatory excesses. This compact, normalized shape is convenient for the pricing calculators to join and iterate in code, and it is what the 39 tests pin.

The second, assar_info.db, inverts the design: one cleanly named SQL table per table in the manual, 46 in total. A request such as "fire rates for the different risks" corresponds directly to the fire_allied_perils table and its risk_category column. Importantly, the assistant does not write SQL from the user's words; Section 11 explains why that pattern is avoided. Instead, these descriptive names serve two safe purposes: they let the assistant's table matcher map a plain-language request to the right table (which is then rendered with a fixed, read-only query), and they keep the Database tab self-documenting for an analyst browsing the data directly. The clean schema is, in effect, its own documentation.

Three further choices shape the data. First, numbers were verified before being reused: the rate values were spot-checked cell by cell against the source PDF,

and the information database reuses those verified values rather than re-transcribing them, confining new manual transcription to the large-risk registers. Second, labels in the information database are exact verbatim strings from the manual, including punctuation, so an agent's text matching lines up with the source wording; a build-time guard fails if any label list drifts out of alignment with its verified values. Third, units are made explicit: most columns carry their unit in the column name (rate_pct, rate_per_mille, insured_value_rwf), and a data_dictionary table documents the unit of every column, so a percentage is never mistaken for a per-mille rate, the easiest order-of-magnitude error in this manual.

---

4. Chunking Strategy

The prose corpus is chunked for retrieval with a structure-aware, section-based strategy rather than a fixed page or character window, because the manual is a long, structured document whose logical units are sections that often span page breaks.

The corpus builder extracts page text into a Markdown corpus, tagging each page so chunks can be cited. The chunker then detects heading runs, the manual's section titles such as "PRICING OF CONSEQUENTIAL LOSS" or "FULL VALUE BASIS", including titles that wrap onto a second line, and groups each section's body together even across page boundaries. Long sections are sub-split on sentence boundaries at roughly 1,600 characters with 200 characters of overlap. Every chunk is prefixed with its section title, which gives the embedding model context about what the chunk is, and carries its page anchor for citation plus the section name in metadata. Front-matter such as the cover page and table of contents is dropped, and stray PDF bullet glyphs are stripped.

The most important chunking decision is what is excluded: the numeric rate tables are not embedded at all. They live in SQLite and are served exactly. This removes the single biggest failure mode for a financial document, an embedding fuzzing a value like 0.3144 percent, and it is why the corpus holds only definitions, conditions, warranties, and guidance.

---

5. Retrieval Strategy

Concept questions are answered by retrieval-augmented generation over the prose corpus, through a staged pipeline whose individual levers can be toggled and measured; the reasoning behind each is documented in docs/RETRIEVAL.md. Chunks are embedded locally with a sentence-transformers model (BAAI/bge-base-en-v1.5; a multilingual model is a drop-in for Kinyarwanda or French) and stored in a persistent ChromaDB collection using cosine similarity.

Retrieval combines three stages. A hybrid first stage runs dense vector search and BM25 keyword search over the same chunks and fuses their rankings with Reciprocal Rank Fusion, so meaning-based matches and exact-term matches (an acronym such as ICC-A, or a word such as reinsurance) both surface. A cross-encoder reranker (ms-marco-MiniLM-L-6-v2) then re-scores the shortlist by reading the question and each passage together, which lifts the single most relevant passage to the top. An optional multi-query expansion stage can rewrite the question into paraphrases; it is off by default because it spends model tokens. Each stage is an environment toggle, and quality is measured rather than assumed: an evaluation harness reports recall@k and MRR@k for the dense, hybrid, and reranked configurations over a labelled question set. The retrieved section title and page travel with each result, so an answer can point the reader to the exact place in the manual.

The router itself is also hybrid by question type. Number and quote questions go to the Quant layer, which calls a typed pricing calculator. A request for a table or comparison is matched to one of the manual's tables in the information database, using the same embeddings and reading recent conversation turns, and rendered directly from SQL. Concept questions go to the retriever. Lookups are made robust with scheme-scoped fuzzy matching, so an approximate category such as "hotel" resolves to the valid key, and the dispatcher coerces string numbers and drops arguments a calculator does not accept.

Grounding is enforced in code, not left to the model: no rate, percentage or amount is shown unless that exact figure appears in a calculator result or a retrieved passage, and anything ungrounded is replaced with an honest request for

the specific cover and sum insured. The model is restricted to general (non-life) business and declines motor, life, or medical questions rather than inventing a quote. The conversation is multi-turn: prior turns are passed back so a follow-up like "and for a bank?" keeps context.

---

6. Audit: SQL Tables Mapped to the Source Manual

The table below maps every table in the information database (assar_info.db) to the table or section it was built from in the ASSAR manual and the page(s) where that source appears, so the data can be verified against the PDF. SQL names are exactly as they appear in the database.

SQL TABLE	PAGE(S)	MANUAL TABLE / SOURCE
voluntary_deductible_discount	p.13	Schedule of Approved Discounts for Voluntary Deductible
special_perils	p.15	Special Perils and Respective Perils Premium Rate
short_period_scale	p.16	Scale of Rates for Short Period Insurance
fire_allied_perils	p.19-22	Fire and Allied Perils Insurance - Commercial/Administrative (Material Damage)
fire_private_dwellings	p.22	Fire Private Dwellings (All Perils Inclusive)
plate_glass	p.22	Pricing of Plate Glass Insurance
consequential_loss_basis	p.24	Rating for Consequential Loss - basis items (Gross Profit / Auditors Fees / Wages)
consequential_loss_indemnity_period	p.24	Indemnity Period Selected - Percentages of the Basis Rate
business_interruption_time_excess	p.24	Voluntary Time Excess under Business Interruption
consequential_loss_dual_basis_wages	p.26	Consequential Loss Table for Dual Basis Wages Cover (manual notes: not commonly used)
burglary_full_value	p.28	Burglary & Theft - Full Value Basis minimum rates
first_loss_multiplier	p.28	Risks Insured on First Loss Basis - multipliers
bankers_blanket_bond	p.31	Bankers Blanket Bond - Description of Risk / Rate
directors_officers_liability	p.32	Directors and Officers Liability - Description of Risk / Rate
money_insurance	p.33-34	Money and Cash in Transit - rates (in safe, ATM, out of safe, custody)
money_annual_carryings	p.33	Money in Transit - Annual Carryings rate bands
goods_in_transit	p.36-38	Goods in Transit - commodity rate grid
transporters_liability	p.42-44	Transporters Liability - commodity rate grid
public_liability	p.46	Public Liability - Minimum Premium Rate
employers_liability	p.47	Employers' Liability - Minimum Premium Rate
school_liability	p.47-48	School Liability - premiums and indemnity limits
school_liability_short_period	p.48	Short Rates for School Liability
product_liability	p.49	Product Liability - Minimum Premium Rate
professional_indemnity	p.50	Professional Indemnity - Professional Classification / Rate
personal_accident_gpa	p.51	PA & GPA Risks Categories and Minimum Premium

		/ Rates
personal_accident_short_period	p.52	Short Rates for Personal and Group Personal Accident
erection_all_risks	p.53-54	Erection All Risks (EAR) - Minimum Rate
machinery_breakdown	p.55-56	Machinery Breakdown - Minimum Rates
cpm_rates	p.57	Contractors Plant & Machinery (CPM) - Hazard Class x Plant Group
cpm_short_period	p.57-58	Short Period Rates under CPM
boilers_pressure_vessels	p.58	Boilers and Pressure Vessels - Material Damage / TPL
eear_computer_all_risks	p.58-59	Computer and Electric & Electronic All Risks (EEAR)
contractors_all_risks	p.62-63	Contractors' All Risks - Minimum Rate
aviation	p.64	Pricing of Aviation Risks - Description of Risk / Applicable Rate
marine_hull	p.66	Rates for Vessels / Marine Hull rating procedure
marine_hull_occupant_premiums	p.66	Premiums and Sums Insured for 1 Occupant (Bodily Injuries)
marine_cargo	p.67-69	Rates for Marine Cargo (ICC-A)
bonds_guarantees	p.70	Pricing of Bonds/Guarantees - Description of Bond / Rate
fidelity_guarantee	p.71	Pricing of Fidelity Guarantee - Description of Risk / Rate
pvt_political_violence_terrorism	p.72-73	Pricing of PVT Risks - Description of Risk / per-mille rate
large_risks_property	p.76-79	List of Large Risks Established in the Market - Property
large_risks_engineering	p.80	List of Large Risks - Engineering
large_risks_accident	p.80	List of Large Risks - Accident Classes
market_parameters	p.12-75	Derived: policy fee (p.12), FEA discount (p.16), stock declaration (p.29), commission (p.75)
minimum_premiums	multiple	Derived: minimum premiums across classes (money p.34, liabilities p.46-49, PA/GPA p.52, bonds p.71, fidelity p.72)
data_dictionary	n/a	Engine metadata - documents the unit of every column (not a manual table)

Total: 46 tables.

Audit notes. Pricing values are reused from the verified seed used by the calculators; the information database adds verbatim labels and the large-risk registers. Three tables are derived rather than transcribed from a single table: market_parameters gathers scattered market-wide figures, minimum_premiums gathers per-class minimums, and data_dictionary is engine metadata documenting units. PVT rates are stored in per mille; every other class is percent.

---

7. Deployment and User Interface

The system runs as a Streamlit web application launched with a single command, backed by the two SQLite files and a local ChromaDB store. Setup installs the requirements, copies the environment template, and adds a language-model key

(Groq by default; the key lives in a gitignored .env). The data is built in three steps: the pricing database from the transcribed tables, the information database (one table per manual table), and the prose corpus plus its vector store, after which the app is started. The embedding model is pre-warmed at startup so the first chat message is not a silent wait. The databases are committed so the project runs out of the box.

The interface is a wide layout with a themed banner and a sidebar that shows live status: the rate database and its row count, the information engine and its 46 tables, the language-model backend and whether a key is set, and whether the vector store is built. A standing caution reminds users to verify rates against the source manual before binding cover.

---

8. What It Does, Section by Section

Chat. The primary, client-facing surface is a multi-turn chat with conversation memory and a strip of product tiles showing the classes covered. A question is classified and routed by question type. Quote answers show the full working inline as a step-by-step table (sum insured, rate, gross, discounts, net, policy fee, final), so an underwriter or client can follow the arithmetic without opening anything; the evidence is the rate table read by the calculator. A request for a table or comparison is answered by matching it to one of the manual's tables and rendering it from SQL, with the manual's verbatim labels and correct units, which lets an underwriter compare rates across risks side by side. Concept answers are grounded in retrieved manual passages shown with page citations. No rate or amount is ever shown unless it is grounded in a calculator result or a retrieved passage; an ungrounded figure is replaced with a request for the specific cover and sum insured, and if a sum insured is missing the assistant states the rate and asks for the amount rather than assuming one. Because it remembers the conversation, follow-up questions work naturally. It can quote every class in the manual that has a calculator, twenty-one tools in all; out-of-scope questions are politely declined.

Get a Quote. A deterministic calculator with dropdowns for the main products. It bypasses the language model entirely and calls the typed calculators directly,

so it always works and always returns exact figures with a full breakdown and the applicable excess. This is the reliable path for a precise quote.

Database. A transparency surface that lets a user browse and run read-only SQL against either database: the four-table pricing engine, or the 46-table information engine with its example queries and the data_dictionary of units. Results can be filtered, searched, and downloaded as CSV. Only SELECT and WITH queries are allowed, on a genuinely read-only connection.

Underneath all three, the pricing calculators read exact rates from SQLite and compose premiums with minimum-premium floors, mandatory excesses, short-period factors, and policy fees as the manual specifies, and they are covered by deterministic tests so a wrong cell or a broken composition is caught immediately. The whole system is designed to be deterministic where the numbers must be exact, fluent where the questions are open-ended, and auditable throughout. As always, the figures should be verified against the source manual before binding cover.

---

9. Pricing Models for the Main Products (Get a Quote)

Every premium on the Get a Quote tab is produced by a typed Python calculator that reads exact rates from SQLite; no language model is involved. The calculators share a common skeleton, then each product applies its own loadings, discounts, and floors as the manual specifies.

```
premium = sum_insured x rate            (rate /100 for percent, /1000 for per mille)
        + loadings (e.g. industrial process load)
        - discounts (FEA, voluntary deductible, security, clause/mode, stock)
        x multipliers (first-loss, multi-trip, indemnity period, duration load)
        x short-period factor            (fraction of annual premium)
        floored at the class minimum premium
        + policy fee (Rwf5,000, where the class charges one)
        = final premium (net of taxes)
```

Fire and Allied Perils

The base rate is the occupancy rate from the fire grid: the standard-fire column (fire, lightning, explosion) or the fire-and-all-special-perils column. The fire portion is sum insured times that rate. Industrial risks add a 0.025 percent process loading plus a flat Rwf25,000 extensions loading, neither of which

enjoys the FEA discount. A 15 percent Fire Extinguishing Appliances discount applies to the fire portion only. A voluntary deductible then earns a banded discount (capped at 33.33 percent of the excess amount). A short-period factor and the policy fee complete the premium. All fire material-damage cover is subject to the Condition of Average.

Public, Employers, Product, and Professional Liability

Premium is the selected limit of indemnity times the occupation rate, adjusted by a short-period factor and floored at the class minimum premium (Rwf100,000 for public, employers, and product; Rwf200,000 for professional indemnity, or Rwf25,000 for insurance agents). Professional indemnity carries a 5 percent mandatory excess (minimum Rwf200,000).

Goods in Transit and Transporters Liability

The base rate comes from the commodity grid, selected by cover type (all-risks or road-accident-only) and whether the cargo is containerized. Transporters liability outside Rwanda loads the rate by 30 percent. For multiple trips the annual premium is scaled by trip period (30 percent up to 3 months, 60 percent up to 6, 90 percent up to 9, 100 percent up to 12). A policy fee is added.

Marine Cargo

The base is the Institute Cargo Clause A rate for the commodity and packing. A transit-mode discount is applied first (road minus 10 percent, sea minus 20 percent, air minus 30 percent, combined none), then a clause discount (Clause B minus 25 percent, Clause C minus 35 percent, Clause A none). A policy fee is added.

Personal Accident and Group Personal Accident

Each benefit is priced off the class base rate: death and total permanent disability at the base rate, total temporary disability at 15 percent of it, and medical and funeral expenses at ten times it. The selected benefits are summed against the capital sum, a short-period factor is applied, and the result is floored at the minimum premium (PA Rwf25,000, or Rwf15,000 for an interning student; GPA Rwf50,000, or Rwf30,000 for a student).

Bonds and Guarantees

Premium is the bond value times the bond-type rate. Providing 100 percent cash collateral reduces the rate to 3 percent. The premium is floored at Rwf10,000 for a bid bond and Rwf30,000 for other bonds. Bonds carry the full annual rate for any period: no short-period or pro-rata reduction applies.

Political Violence and Terrorism (PVT)

PVT rates are quoted per mille, not percent, so the premium is sum insured times the rate divided by one thousand. Approved security features can earn a discount of up to 10 percent. The mandatory deductible is 5 percent of each loss, minimum 0.5 percent of the sum insured, with a floor of Rwf50,000.

Engineering: Contractors All Risks and Erection All Risks

Premium is the contract value times the project-type rate. Projects running beyond twelve months are loaded by 25 percent for each additional six-month block. Third-party liability is included while its limit stays within 15 percent of the contract value; above that it is rated separately at 0.2 percent. A policy fee is added.

Machinery Breakdown

Premium is the sum insured times the machine or industry rate, plus a policy fee. The mandatory excess is 10 percent of each loss (minimum Rwf500,000) for sums insured above Rwf5,000,000, otherwise 5 percent (minimum Rwf250,000).

Contractors Plant and Machinery (CPM)

The rate is read from the hazard-class by plant-group matrix: plant groups are cranes (1), mobile plant (2), and non-mobile plant (3); hazard classes A, B, and C reflect terrain and exposure. A CPM-specific short-period scale and a policy fee apply. The mandatory excess is 10 percent of claim, minimum Rwf500,000.

Burglary and Theft

The full-value rate is 0.3 percent for ordinary goods and 0.5 percent for high-value goods. On a first-loss basis a multiplier from the first-loss schedule is applied according to the ratio of first-loss sum insured to full

value. A stock-declaration basis earns a 10 percent discount. A short-period factor and policy fee apply; the mandatory excess is 10 percent of each loss, minimum Rwf50,000.

Source anchors for the pricing rules

Each rule above is taken from a specific point in the manual; the page(s) are listed below so the methodology can be verified against the ASSAR document. This covers the pricing rules and loadings; the underlying rate values are spot-checked against the manual and should be confirmed cell by cell before binding cover.

PRICING RULE	ASSAR PAGE
Fire: standard-fire vs fire+all-special-perils columns	p.19-22
Fire: industrial +0.025% process load + flat Rwf25,000 extensions	p.17
Fire: 15% FEA discount on fire/lightning/explosion only	p.16
Voluntary-deductible saving capped at 33.33% of the excess amount	p.13
Fire material damage subject to the Condition of Average	p.18
Liability minimum premiums (100k; PI 200k; agents 25k); PI excess 5% min 200k	p.47-50
Transporters +30% outside Rwanda; multi-trip 30/60/90/100% of annual	p.38-41
Marine mode discount (road 10 / sea 20 / air 30); clause B 25, C 35	p.69
PA/GPA: death=TPD; TTD 15%; medical & funeral 10x death; class minimums	p.51-52
Bonds: 100% cash collateral -> 3%; min bid 10k / other 30k; no short period	p.70-71
PVT per mille; security discount <=10%; deductible 5% min 0.5% of SI, floor 50k	p.72-74
Engineering: +25% per extra 6 months; TPL <=15% included, else 0.2% separately	p.55,63
Machinery excess 10% min 500k above 5M SI, else 5% min 250k	p.56
CPM hazard-class x plant-group matrix; excess 10% of claim, min 500k	p.57
Burglary 0.3%/0.5% full value; first-loss multipliers; stock declaration -10%	p.28-29

---

10. Calculation Functions Reference

This section explains every function in the pricing engine (assar/pricing/), so the project can be understood and maintained end to end. There are no hard-coded answers anywhere: each premium is computed from exact rates read from SQLite.

Shared building blocks (base.py)

Every calculator is assembled from a few primitives.

- Quote: the result object. It carries the product, sum insured, effective rate and unit, gross/net/final premium, policy fee, excess, a list of human-readable breakdown lines, and any warnings.

- get_rate(scheme, category, alt): exact rate lookup from the rate table; on an exact miss it falls back to a scheme-scoped fuzzy match (so "hotel" resolves to a valid key). alt selects the second column, e.g. fire's all-perils rate.
- premium_from_rate(sum_insured, rate, unit): the core multiply. Percent divides by 100, per mille by 1000, and an "amount" unit returns the flat figure.
- voluntary_deductible_discount(excess, gross): the banded discount for a chosen excess, with the saving capped at 33.33 percent of the excess amount.
- short_period_fraction(period, schedule): the fraction of the annual premium for cover shorter than a year; 1.0 for a full year.
- apply_minimum, product_rule, policy_fee: floor a premium at the class minimum, read per-product constants (minimums, excesses, loadings), and the Rwf5,000 policy fee.

Fire family (fire.py)

- quote_fire: base rate from the fire grid (standard fire or all-special-perils column); adds the 0.025 percent industrial process load and Rwf25,000 extensions for industrial risks; applies the 15 percent FEA discount to the fire portion only; applies the voluntary-deductible discount; short-period factor; policy fee. Flags the Condition of Average.
- quote_consequential_loss: uses the fire material-damage rate as the basis, times the cover factor (gross profit 150, auditors 125, wages 100 percent), times the indemnity-period multiplier; short period; policy fee; 14-day excess.
- quote_burglary: full-value rate 0.3 percent ordinary or 0.5 percent high-value; first-loss multiplier when a first-loss ratio is given; 10 percent stock-declaration discount; short period; fee; excess 10 percent min Rwf50,000.

Liability, accident, engineering, specialty (products.py)

- quote_liability: limit of indemnity times the occupation rate; short period; minimum premium (100k; professional indemnity 200k, or 25k for agents); professional indemnity carries a 5 percent excess.
- quote_pa_gpa: prices each selected benefit off the class base rate (death and TPD at base, TTD at 15 percent, medical and funeral at ten times base), sums them against the capital sum; short period; PA/GPA minimums.
- quote_bond: bond value times the bond-type rate; 100 percent cash collateral reduces the rate to 3 percent; minimum (bid 10k, other 30k); full annual rate, no short period.

- quote_pvt: per-mille rate (divide by 1000); security-features discount up to 10 percent; mandatory deductible 5 percent, min 0.5 percent of SI, floor 50k.
- quote_car_ear: contract value times project rate; plus 25 percent for each extra six months beyond twelve; TPL included if within 15 percent of the value, else rated separately at 0.2 percent; policy fee.
- quote_machinery: sum insured times the machine/industry rate; policy fee; excess tiered at the Rwf5,000,000 sum-insured threshold.
- quote_cpm: rate from the hazard-class by plant-group matrix; CPM short-period scale; policy fee; excess 10 percent min Rwf500,000.
- quote_fidelity: sum insured times rate (or Rwf30,000 per employee for blanket cover); short period; min 200k; excess Rwf250,000 or 10 percent.
- quote_bbb: bankers blanket bond, limit times 5 percent; excess 250k or 10 percent.
- quote_do_liability: directors and officers, limit times rate (financial services 5 percent, other offices 2.5 percent); excess 250k or 10 percent.
- quote_school_liability: flat premium per student times the number of students; short period; inclusive of policy fees and VAT.
- quote_aviation: sum insured times the class rate; for passenger cover, multiplied by the number of seats; policy fee.
- quote_marine_hull: hull all risks at 0.8 percent of vessel value, or third-party liability at 0.25 percent; policy fee.
- quote_boiler: material damage or third-party liability at 0.5 percent; policy fee; excess 10 percent min Rwf625,000.
- quote_eear: computer/electronic all risks, 0.75 percent at premises, 2 percent portable away, 1.5 percent unspecified tender, 0.75 percent increased cost of working; policy fee; excess 10 percent min Rwf100,000.
- quote_plate_glass: sum insured times 2 percent; policy fee; excess 5 percent min Rwf100,000.

Transit (transit.py)

- _transit_rate (helper): selects the correct cell from the commodity grid (road-accident vs all-risks, containerized vs not), with a fuzzy commodity fallback; returns the rate and the excess text.
- quote_git: Goods in Transit / Transporters Liability; base grid rate; plus 30 percent if transporters liability outside Rwanda; multi-trip scaling (30/60/90/100 percent by period); policy fee.
- quote_marine_cargo: ICC-A base rate; applies the transit-mode discount (road

10, sea 20, air 30 percent), then the clause discount (B 25, C 35 percent); policy fee.

The dispatcher (registry.py)

- run_tool(name, args): the single entry point the chat uses. It coerces string numbers to real numbers, drops any argument a calculator does not accept (so a stray model-generated argument cannot crash a quote), dispatches to the right function, and returns the quote as a dictionary. Errors are returned as data rather than raised. The same file defines the LLM tool schemas that expose all 21 calculators to the chat.

---

11. Security: the Model Never Writes SQL

A common but dangerous pattern in AI data assistants is text-to-SQL: the language model writes raw SQL from the user's words and that SQL is executed against the database. This approach is vulnerable in ways that cannot be fixed at the prompt level. A crafted message can steer the model into destructive statements such as DROP TABLE or UPDATE rates to zero. On a multi-insurer platform it can also cross authorization boundaries, for example returning one insurer's private rate overrides to another. System-prompt instructions such as "never write destructive SQL" are advisory, not enforceable, and prompt-injection attacks are designed to override them. A read-only database user blocks the writes but does not stop a read from crossing a tenant boundary, because read-only is not the same as authorized.

This system does not use that pattern. The model never composes or executes SQL. Its only job on a pricing turn is to read the request and fill the typed parameters of a pre-defined tool; our code runs the database access. There are three controlled surfaces, and SQL is confined to code we wrote.

First, the pricing tools. The chat exposes the calculators through typed tool schemas (the 21 functions in registry.py), each with named, typed parameters: a sum insured is a number, a cover is drawn from a fixed list, and so on. The dispatcher run_tool validates and coerces the arguments, drops any argument a calculator does not accept, and calls the deterministic Python function. Those

functions read rates with bound SQL parameters and fixed identifier whitelists (the column is chosen from a known set, never built from user text), so even our own queries are not assembled by string-concatenating user input. The model supplies values, not SQL.

Second, the comparison tables. A request for a table is matched to one of the manual's tables by name, and that name is validated against the database's actual table list before anything runs. The render is a fixed read against that whitelisted table. The model selects which table, never how to query it.

Third, the database browser. The only place SQL is typed is the Database tab, where a human, not the model, types it. That path uses a genuinely read-only SQLite connection (opened in mode=ro), allows only a single SELECT or WITH statement, rejects multiple statements, and blocks write and DDL keywords (insert, update, delete, drop, alter, create, attach, detach, pragma, replace, vacuum). The model is not in this loop at all.

Reinforcing all three, the grounding guard ensures no rate, percentage, or amount reaches the user unless it came from a tool result or a retrieved manual excerpt, so the model cannot present a number it invented even in prose.

This is the same separation that the Model Context Protocol (MCP) formalizes: the intelligence stays with the model, which chooses a tool and fills its parameters, while access control stays on the server, which owns the query. Each typed tool here is already the natural unit to publish as an MCP tool or a thin endpoint wrapper, with no change to the security model.

The current prototype prices the single shared ASSAR approved schedule, so it has no tenant isolation yet. Multi-insurer tenancy and authorization, in particular per-insurer rate overrides scoped to the authenticated insurer, are the planned next step, and the tool-and-endpoint design above is exactly what makes that enforceable: every tool call can be bound to the caller's identity on the server, which model-written SQL could never guarantee.

End of report.